Artifical Neural Networks and Deep Learning
Homework 1:

Axel Friberg Hagen
Vegard Sjåvik
Lasse Uttian

November 17, 2023

1 Foundational Overview: Introduction and Motivation

In this project, we explore the task of classifying healthy and unhealthy plants using convolutional neural networks. By utilizing a diverse (5200 samples) dataset of labeled *plant* images, our goal was to develop a model capable of accurately distinguishing between the two classes.

In the following sections, we will discuss our methodology, data preprocessing, model architecture, training process, and evaluation results. By the end of the paper, the readers will gain insights into our thought-processes and techniques when facing a problem like this one.

2 Evolution of Ideas: Experimental Insights

This part of the report is going to address the different experimentations that were done to figure out which model to optimize with hyperparameter search.

2.1 Base model - LeNet model

The process was started by making a convolutional neural network based on the LeNet architecture. This model quickly convereged to an accuracy of 0.6, which is not better than a classifier guessing the majority label for all images. This underperformance is assumed to be due to the shallow architecture (and few parameters) of the model, and the lack of data needed to train robust features [2].

2.2 Strategic Pivot - Transfer learning

To improve the model performance, transfer learning was employed instead. This involved using a pre-trained model on the ImageNet dataset for feature extraction, removing the final layer and replacing it with a custom classification layer. To assess which pre-trained model would best fit our problem, several models where tested against the same validation set with the same hyperparameters.

Model	Accuracy	AUC	F1
MobileNetV2	0.81	0.87	0.79
ResNet	0.63	0.69	0.57
InceptionV3	0.38	0.52	0.30
VGG16	0.63	0.59	0.39

Table 1: Performance Metrics of Different Models on the validation set

As Table 1 shows, MobileNetV2 emerged as the most promising model, demonstrating superior performance in terms of accuracy, AUC (Area Under the Curve) and F1 score. Given these results, the focus of the project shifted towards optimizing the MobileNetV2 model. This decision is also backed by the fact that MobileNetV2 requires few data points to converge [1].

2.3 Addressing underfitting - Finetuning model

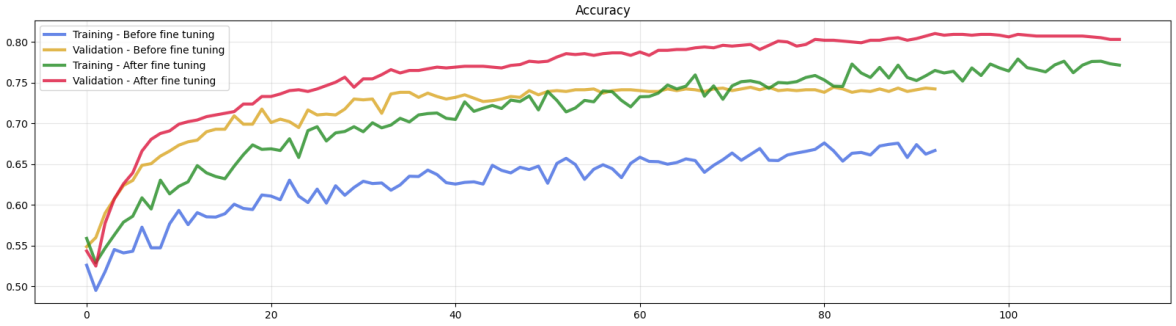


Figure 1: Comparison between our model before and after finetuning.

Figure 1 illustrates that the initial model underperforms on the training and validation sets, indicativing underfitting [3]. To make up for this, we employed fine-tuning techniques. The results of this approach are depicted in Figure 1, which reveals significant improvements in both training and validation accuracy, alongside enhanced coherence between them.

2.4 Experimentation for minor improvements

2.4.1 Data-cleaning

After some data-exploration - pictures of shrek and trololo where found in addition to several duplicates. The shrek/trololo images were removed for obvious reasons, while the duplicates were removed to let the network generalize better and learn more diverse features, reducing overfitting.

2.4.2 Image augmentation

To increase the size of the dataset, as well as to force the model to learn more robust features, images where augmented with the following augmentations: Random brightness, random translation and rotation, contrast modification and flipping. This addition proved to reduce the error on the validation set. We choose not to use zooming-augmentation as we thought it could ”cut out” the damaged part of the plant, effectively creating false data (which would be very bad for our model).

2.4.3 Dealing with class imbalance

Initially, an undersampling strategy was employed to balance class distributions by reducing the size of the majority class. However, due to limited data availability, this method proved to be ineffective. Instead, we tried implementing a weighted loss function by modifying the Binary Cross-Entropy (BCE). Additionally, adjusting the classification threshold was explored. Yet, after hyperparameter-searching, it became evident that the optimal threshold remained at 50%, which evidently became obvious as the weighted BCE would naturally converge to a 50/50 equilibrium.

2.4.4 Addressing overfitting

We implemented dropout in the MLP to promote reduced dependency on specific weights, aiming to reduce overfitting on the test set.
We then lowered the batch-size from 256 to 32, introducing more stable (but potentially slower) training.
Finally an early-stopping callback was added to the model to prevent overfitting, as continued training might fit more noise to the model without improving accuracy. After an extensive search it was established that a patience of 20 in this callback was sufficient.

2.4.5 Addressing preprocessing issues

To better fit the inputs of MobileNetV2, the data was normalized in the range -1 to 1. However, this significantly decreased our training scores. Hence the 0 to 1 normalization was kept.

3 The Culminating Model: Final Design and Structure

Figure 2 shows the architecture of the final model. The model had the following parameters: Dropout rate of 0.5, learning rate of $1E-5$, batchsize to 128, 200 epochs and early stopping with patience 20.

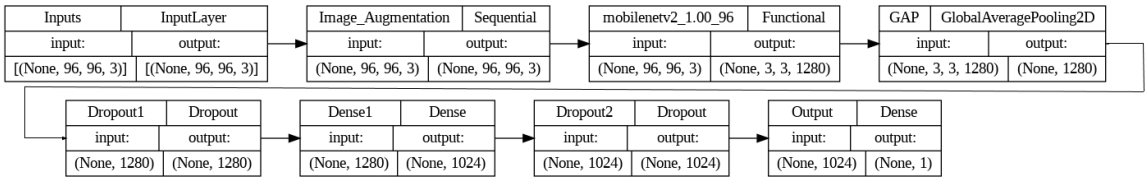


Figure 2: Comparison between our model before and after finetuning.

4 Future Frontiers: Unimplemented and Untested Ideas

- Learning rate rescheduler. For example an exponetional Different loss functions, such as Focal Loss, Dice Loss or Tversky Loss.
- Regularizing the weights by adding a penalty term (L1 or L2-norm of weights for example) in the loss function.
- Batch-normalization was not used as its usefulness in transfer-learning is not well-established. Could still be interesting to explore.
- Dropout in the convolutional layers can be very effective in reducing overfitting, but was not explored as we were not using any manual convolutional layers, but relied entirely on the pre-trained model.

5 Contributions

Contributors

Axel Hagen:

- Sorted out Shrek/Trololo images from the dataset.
- Removed duplicate images from the dataset.
- Explored and suggested various ideas for improvement; judged and decided which ideas to pursue.
- Wrote the project report.
- Engaged in extensive research and discussions with the team.

Lasse Uttian:

- Unfortunately, Lasse has been unable to contribute due to health issues, but his portion of the work was gladly carried out by the rest of the team.

Vegard Sjåvik:

- Explored different models and conducted a significant hyperparameter search to enhance model performance, incorporating augmentation techniques.
- Conducted research on transfer learning and fine-tuning.
- Fixed submission files.
- Wrote the project report.
- Engaged in substantial research and discussions with the team.

References

[1] Alfredo Canziani, Adam Paszke, and Eugenio Culurciello. An analysis of deep neural network models for practical applications, 2017.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[3] IBM. Underfitting, 2023.